

UIKit → SwiftUI

Button { }

@State

@IBOutlet

How every pattern in ViewController maps to SwiftUI · ElementQuiz app

① STATE — HOLDING DATA

UIKit @IBOutlet

```
@IBOutlet weak var
answerLabel: UILabel!
Pointer to storyboard element
```

SwiftUI @State

maps to **@State** private var
isAnswerShown = false
Changing this triggers a redraw

② ACTIONS — RESPONDING TO TAPS

UIKit @IBAction

```
@IBAction func showAnswer(
_ sender: Any) {
    answerLabel.text = "Carbon"
}
Wired in Interface Builder
```

SwiftUI Button closure

maps to **Button("Show Answer") {**
isAnswerShown = true
}
Action lives next to the button

③ UPDATES — REFRESHING THE UI

UIKit updateElement()

```
func updateElement() {
    imageView.image = UIImage(...)
    answerLabel.text = "?"
}
Called manually after every change
```

SwiftUI declarative view

gone **Image(currentElement)**
Text(isAnswerShown ? currentElement : "?")
View always reflects current state

④ INDEX WRAPPING — CYCLING ELEMENTS

UIKit if / reset

```
index += 1
if index >= list.count {
    index = 0
}
Explicit boundary check
```

SwiftUI % modulo

maps to **index = (index + 1)**
% list.count
% wraps automatically — cleaner Swift

The core mental shift

UIKit is imperative — you command the UI: "set this label's text to X."

SwiftUI is declarative — you describe what to show: "show the answer when isAnswerShown is true."

Change a @State var, and SwiftUI handles the redraw automatically.

QUICK REFERENCE

UIKit

```
@IBOutlet
@IBAction func
viewDidLoad()
updateElement()
if index >= count
```

SwiftUI

```
@State
Button { }
Initial @State value
(remove)
% count
```

What changed

Data replaces UI pointer
Closure replaces method
No lifecycle hook needed
SwiftUI auto-redraws
Modulo is cleaner